字符串

1. 基础定义 1
1.1. 子串,前缀(prefixs),后缀(suffixs) 1
1.2. 周期 1
1.2.1. 周期定理 1
1.3. 循环节 1
1.4. Border 1
1.4.1. 特殊性质 1
2. 哈希 2
2.1. 单哈希 2
2.2. 双哈希 3
2.3. 带修哈希 4
2.4. 树哈希 4
3. KMP 4
3.1. 原理 4
3.2. kmp 求 border 4
3.3. kmp 匹配 4
4. Border 树 5
4.1. 定义 5
4.2. 性质 5
5. Manacher 5

1. 基础定义

1.1. 子串,前缀(prefixs),后缀(suffixs)

1.2. 周期

存在一个p,使得$p < i \leq |S|$,满足$S[i] = S[i-p]$,则称 p 是 S 的一个周期

1.2.1. 周期定理

如果p, q均是s的周期,则$\gcd(p, q)$也是 s 的周期

1.3. 循环节

若周期$p \mid |S|$,则周期 p 是 S 的一个循环节

1.4. Border

存在一个长度 len,满足该长度的前缀和后缀相同,则称该前缀/后缀是该字符串的一个 border

1.4.1. 特殊性质

1. p 是 S 的周期⇔|S|-p 是 S 的 Border

证明:

p 是 S 的周期⇔S[i-p]=S[i]

p 是 S 的 Border⇔S[1,q]=S[|S|-q+1,|S|]

$S[1] = S[|S| - q + 1]$

$S[2] = S[|S| - q + 2]$

2. Border 的传递性

S 的 Border 的 Border 的也是 S 的 Border

3. 一个串的 Border 数量是$O(N)$个,但是他们组成了$O(\log N)$个等差数列

2. 哈希

2.1. 单哈希

```
struct Hash
{
    const ll base = 1361;
    const ll p = 998244353;
    int n;
    vector<ll> a, c;
    Hash(int _n, string arr) : n(_n), a(n + 10), c(n + 10, 0)
    {
        ll res = 0;
        c[0] = 1;
        for (int i = 1; i <= n; i++)
        {
            res = (res * base + arr[i]) % p;
            c[i] = c[i - 1] * base % p;
            a[i] = res;
        }
    }
    ll query(int l, int r) // 询问子串的哈希值
    {
        // if (l < 1 || r > n)
        //     cout << "invalid input" << endl;
        ll ans = (a[r] - a[l - 1] * c[r - l + 1] % p + p) % p;
        return ans;
    }
```

```
};
```

2.2. 双哈希

```
struct Hash
{
    const ll base1 = 1361, base2 = 1301;
    const ll p1 = 1000000123, p2 = 1000000021;
    int n;
    vector<ll> a1, a2, c1, c2;
    Hash(int _n) : n(_n), a1(n + 10), a2(n + 10), c1(n + 10, 0), c2(n + 10) {}
    void build(int n, vector<char> arr)
    {
        ll res = 0;
        c1[0] = 1, c2[0] = 1;
        for (int i = 1; i <= n; i++)
        {
            res = (res * base1 + arr[i]) % p1;
            c1[i] = c1[i - 1] * base1 % p1;
            a1[i] = res;
        }
        res = 0;
        for (int i = 1; i <= n; i++)
        {
            res = (res * base2 + arr[i]) % p2;
            c2[i] = c2[i - 1] * base2 % p2;
            a2[i] = res;
        }
    }
    pll query(int l, int r) // 询问子串的哈希值
    {
        ll hashval1 = (a1[r] - 1ll * a1[l - 1] * c1[r - l + 1] % p1 + p1) % p1;
        ll hashval2 = (a2[r] - 1ll * a2[l - 1] * c2[r - l + 1] % p2 + p2) % p2;
        return {hashval1, hashval2};
    }
};
```

2.3. 带修哈希

用树状数组维护即可

```
1
```

2.4. 树哈希

3. KMP

3.1. 原理

前缀 i 的 border 的长度-1→前缀 i-1 的 border 的长度

kmp 实际就是在暴力这个逆过程,枚举右边,看看是否满足

next 数组:

next[i]=Preffix[i]的非平凡的最大 Border(非平凡:除掉自己)

3.2. kmp 求 border

```
vector<int> nxt(N, 0);
void kmp(vector<char> &s, int n)
{
    nxt[1] = 0;
    for (int i = 2; i <= n; i++)
    {
        nxt[i] = nxt[i - 1];
        while (nxt[i] && s[i] != s[nxt[i] + 1])
            nxt[i] = nxt[nxt[i]];
        nxt[i] += (s[i] == s[nxt[i] + 1]);
    }
}
```

3.3. kmp 匹配

```
struct KMP {
    vector<char> t;
    vector<int> nxt, ans;
    int n;
    KMP(int n, string &s1) : n(n), t(n + 10), nxt(n + 10) { // 求 border
        for (int i = 1; i <= n; i++)
            t[i] = s1[i];
```

```
        nxt[1] = 0;
        for (int i = 2; i <= n; i++) {
            nxt[i] = nxt[i - 1];
            while (nxt[i] && t[i] != t[nxt[i] + 1])
                nxt[i] = nxt[nxt[i]];
            nxt[i] += (t[i] == t[nxt[i] + 1]);
        }
    }
    ll find(int m, string &s) { // kmp 匹配
        ll p = 0, res = 0;
        for (int i = 1; i <= m; i++) {
            while (p == n || (p && t[p + 1] != s[i]))
                p = nxt[p];
            p += (t[p + 1] == s[i]);
            if (p == n)
                res++;
        }
        return res;
    }
};
```

4. Border 树

4.1. 定义

对于一个长度为 n 的字符串,它的 border 树有 n+1 个节点

0 是树的根,对于其他节点,父节点为 nxt[i]

4.2. 性质

1.字符串的某个前缀 i 的 border 就是 border 树中 i 到根的链

2.求哪些前缀有长度为 x 的 border:x 的子树

3.求两个前缀的公共 border:border 树的 lca

---

5. Manacher

```
struct Manacher
{
    // 设插入 $ 后的字符串为 s',那么 s'与原串 s 存在以下关系:
    // s'的极长回文子串一定为奇数(因为一定以 $ 结尾)
```

```
    //|s'|=2|s|+1
    //|s|=(|s'|-1)/2
    //|s|=|s'|/2(下取整)
    int m;
    vector<char> a;
    vector<int> p; // p[i]表示以 i 为中心的最长回文子串
    Manacher(int _n) : a(2 * _n + 10), p(2 * _n + 10) {}
    void build(int n, vector<char> &s)
    {
        m = 0;
        a[++m] = '$';
        for (int i = 1; i <= n; i++)
            a[++m] = s[i], a[++m] = '$';
        int M = 0, R = 0;
        for (int i = 1; i <= m; i++)
        {
            if (i > R)
                p[i] = 1;
            else
                p[i] = min(p[2 * M - i], R - i + 1);
            while (i - p[i] > 0 && i + p[i] - 1 <= m && a[i - p[i]] == a[i + p[i]]) // 暴力向右拓展
                ++p[i];
            if (i + p[i] - 1 > R) // 更新右端点最右的回文串的回文中心,回文半径
                M = i, R = i + p[i] - 1;
        }
    }
    int query()
    {
        int ans = 0;
        for (int i = 1; i <= m; i++)
            ans = max(ans, p[i] - 1);
        return ans;
    }
};
```